ASSIGNMENT 02 Full Stack

2. CSS Flexbox Navbar

CSS Flexbox is a CSS layout system used to arrange elements easily in a row or column. In this question, the navbar contains a logo on the left and three menu links on the right. We use display: flex to create the Flexbox layout, justify-content: space-between to push the logo and menu to opposite sides, and align-items: center to keep everything vertically centered.

Answer / Corrected Code:

```
<nav class="navbar">
 <div class="logo">MyShop</div>

 <div class="menu">
  <a href="#">Home</a>
  <a href="#">Products</a>
  <a href="#">Contact</a>
 </div>
</nav>

<style> .navbar
{  display: flex;
 justify-content: space-between;  align-
items: center;  padding: 15px 30px;
background: #333; }

.logo {  color: white;
font-size: 22px;  font-
weight: bold; }

.menu {  display:
flex;  gap: 25px; }

.menu a {  color: white;  text-
decoration: none; }
</style>
```

Explanation: The navbar becomes a Flexbox container. The logo is placed on the left and the menu links are placed on the right. The align-items: center property vertically centers the logo and links. The gap property creates space between the three links.

3. Fixing the Dark-Mode Button

The given button is supposed to add or remove a dark-mode class from the body when the button is clicked. The problem is a spelling mistake in the event-listener method. The correct JavaScript method is addEventListener(), but the given code uses addEventListner().

Answer / Corrected Code:

```
const button = document.querySelector('.toggle-btn');

button.addEventListener('click', () =>
{  document.body.classList.toggle('dark-mode'); });
```

Explanation: addEventListener() attaches the click event to the button. When the user clicks the button, classList.toggle() adds the dark-mode class if it is not present and removes it if it is already present. CSS can then use .dark-mode to change the page colors.

4. JavaScript Signup Form Validation

Form validation checks the information entered by the user before allowing the form to be submitted. In this question, the email must contain the @ symbol and the password must

contain at least 8 characters. The validation is done using normal JavaScript without any library.

Answer / Corrected Code:

```
<form id="signupForm">
  <input type="email" id="email" placeholder="Email">
  <input type="password" id="password" placeholder="Password">
  <button type="submit">Sign Up</button>
</form>

<script> const form = document.getElementById("signupForm");

form.addEventListener("submit", function(event) {   event.preventDefault();

  const email = document.getElementById("email").value;   const password =
document.getElementById("password").value;

  if (!email.includes("@")) {     alert("Please enter a
valid email.");     return;   }

  if (password.length < 8) {     alert("Password must be at least 8
characters.");     return;   }

  alert("Signup form is valid!");
form.submit(); });
</script>
```

Explanation: First, preventDefault() stops the normal form submission. The JavaScript then reads the email and password. includes("@") checks whether the email contains @, while password.length checks the password length. If the email is invalid or the password is shorter than 8 characters, an error message is displayed. If both are valid, the form is submitted.

5. Correct Use of Scroll Event Listener

The original renderPage() function creates a new scroll event listener every time it runs. If renderPage() is called many times, many listeners can remain attached to the window. This means the same work may be performed repeatedly and unnecessary callback functions can remain in memory.

Answer / Corrected Code:

```
function handleScroll()
{   console.log(window.scrollY); }

function renderPage() {   window.addEventListener("scroll",
handleScroll); }

function removePageListener()
{   window.removeEventListener("scroll", handleScroll); }

renderPage();

// When the listener is no longer required:
// removePageListener();
```

Explanation: Repeatedly adding listeners without removing old ones can cause unnecessary memory usage because the old callback functions remain attached. It can also cause the same code to run multiple times for one scroll event. By keeping the handler in a named function, the exact same listener can later be removed using removeEventListener().

6. Debounced Search Input Handler

A search box can generate many input events while a user is typing. Calling the API after every single character can create many unnecessary requests. Debouncing solves this problem by waiting for a short period after the last keystroke. Here, the required waiting time is 400 milliseconds.

Answer / Corrected Code:

```
const searchInput = document.getElementById("searchInput");

let timer;

searchInput.addEventListener("input", function(event) {   const query =
event.target.value;

  clearTimeout(timer);

  timer = setTimeout(() =>
{    searchProducts(query);   }, 400);
});
```

Explanation: clearTimeout() cancels the previous timer whenever the user types again. A new timer is then started. If the user stops typing for 400 milliseconds, the timer finishes and searchProducts(query) is called.
This reduces unnecessary API calls and improves performance.

7. Responsive Layout Below 480px

The original CSS uses fixed widths: the container is 960px wide and each card is 300px wide. A mobile screen below 480px cannot comfortably display such a wide fixed layout, so the content can overflow horizontally. A media query can change the layout for small screens.

Answer / Corrected Code:

```
.container {

  display: flex;
width: 960px;   gap:
20px; }

.card {   width:
300px; }

@media (max-width: 480px)
{   .container {     width: 100%;
flex-direction: column;     padding:
10px;     box-sizing: border-box;   }

  .card {     width: 100%;     box-
sizing: border-box;   }
}
```

Explanation: The media query is applied when the screen width is 480px or less. The container becomes 100% wide and the cards become 100% wide. flex-direction: column places the cards one below another, making the layout suitable for a mobile screen.

8. Saving and Restoring Theme Using localStorage

localStorage is a browser feature used to store small pieces of information. The stored data remains available even after the page is refreshed. We can save the user's selected theme as "light" or "dark" and restore it when the page loads again.

Answer / Corrected Code:

```
const themeButton = document.getElementById("themeButton");

// Restore the saved theme const savedTheme =
localStorage.getItem("theme") || "light";
document.body.classList.add(savedTheme);

// Save a new theme
themeButton.addEventListener("click", () => {
  const isDark = document.body.classList.contains("dark");  const newTheme
= isDark ? "light" : "dark";

  document.body.classList.remove("light", "dark");
document.body.classList.add(newTheme);

  localStorage.setItem("theme", newTheme); });
```

Explanation: localStorage.setItem() saves the selected theme in the browser. On the next page load, localStorage.getItem() reads the saved value. The corresponding class is then added to the body. Therefore, the user's theme remains selected after a page reload.

9. Preventing Form Refresh and Using fetch()

Normally, submitting a form causes the browser to refresh or navigate to another page. This can cause JavaScript state to be lost. To avoid this, preventDefault() stops the browser's default submission behavior, and fetch() is used to send the form data to the server.

Answer / Corrected Code:

```
const form = document.getElementById("contactForm");

form.addEventListener("submit", async (event) => {  event.preventDefault();

  const formData = new FormData(form);

  const response = await fetch("/api/contact", {    method:
"POST",    body: formData  });

  const result = await response.json();

  console.log(result); });
```

Explanation: When the user clicks Send, the submit event occurs. preventDefault() stops the normal page refresh. FormData collects the values from the form, and fetch() sends them to the /api/contact endpoint using a POST request. The response is then converted to JSON.

10. Correcting the async Function

The function is declared as async, but the two asynchronous operations are missing the await keyword.
fetch() returns a Promise, and response.json() also returns a Promise. Therefore, await must be used before both operations to obtain their results.

Answer / Corrected Code:

```
async function getUser(id) {  const res = await fetch(`/api/
users/${id}`);  const data = await res.json();

  return data; }
```

Explanation: The first await waits for fetch() to complete and gives the actual Response object. The second await waits for the response body to be converted into JavaScript data. Finally, the parsed user data is returned. Without await, the variables would contain unresolved Promise objects instead of the required results.

11. Responsive Image Gallery Using CSS Grid

CSS Grid is used to create rows and columns. The question asks for a responsive gallery using minmax() and auto-fit instead of a media query. auto-fit automatically adjusts how many columns can fit in the available space, while minmax() sets the minimum and maximum size of each column.

Answer / Corrected Code:

```
<div class="gallery">
  <img src="image1.jpg" alt="Image 1">
  <img src="image2.jpg" alt="Image 2">
  <img src="image3.jpg" alt="Image 3">
  <img src="image4.jpg" alt="Image 4">
  <img src="image5.jpg" alt="Image 5">
  <img src="image6.jpg" alt="Image 6">
</div>

<style> .gallery {   display:
grid;   grid-template-columns:
    repeat(auto-fit, minmax(250px, 1fr));   gap: 15px; }

.gallery  img {

  width: 100%;   height:
220px;   object-fit: cover;
border-radius: 8px; }
</style>
```